Vibe Studio — Architecture Document

AI-Powered Design-to-Code Platform
Comprehensive System Documentation

Prepared for architectural review

Table of Contents

1. System Overview

2. Technology Stack

3. Project Structure

4. Core Backend Logic

  4.1 Server & Routing

  4.2 Project & Design Pack Services

  4.3 AI Executor — The Iteration Loop

  4.4 Prompt Builder & Response Parser

  4.5 Screenshot Service (Playwright)

  4.6 Visual Scoring Service (Claude Vision)

  4.7 Run Service & Event System

  4.8 Sandbox Manager (Preview Servers)

  4.9 Snapshot Service (Historical Previews)

  4.10 WebSocket Streaming

  4.11 Shared Types & Contracts

5. Core Frontend Logic

  5.1 App Layout & Routing

  5.2 ProjectWorkspace (State Root)

  5.3 Three-Pane Layout

  5.4 Preview Pane & Dev Server Lifecycle

  5.5 Agent Trace Visualization

  5.6 File Tree Explorer

  5.7 Baseline Image Viewer

  5.8 Historical Preview & Click-to-Focus

  5.9 Fullscreen Preview

5.10 WebSocket Hook (useTraceStream)

5.11 API Client

6. Design Elements

6.1 Design Token System

6.2 Component Library

6.3 Layout Patterns

6.4 Typography & Iconography

6.5 Status Indicators

7. Data Flow & Integration

7.1 Design Pack Upload Flow

7.2 Run Execution Flow

7.3 Real-time Event Streaming

7.4 Historical Preview Flow

8. Key Configuration Values

9. File Inventory

1. System Overview

Vibe Studio is an AI-powered design-to-code platform that automatically generates frontend implementations from design specifications. It accepts Design Packs (ZIP files containing a manifest, Design IR, and baseline images), then iteratively generates Next.js code using Claude, screenshots the result with Playwright, scores visual fidelity against the baseline using Claude Vision, and refines until a quality threshold is met.

The system operates as a pnpm monorepo with a Fastify API server, a Next.js web application, and shared packages for types and sandbox management. Real-time progress is streamed via WebSocket, and all trace events are visualized in a hierarchical tree view.

Core Workflow

1. Upload Design Pack (ZIP) → validate manifest + Design IR + baseline images
2. Start Run → AI executor begins iteration loop
3. Each Iteration:
   a. Code Generation: Claude generates Next.js/React code from Design IR
   b. Screenshot: Playwright captures rendered output at each breakpoint
   c. Scoring: Claude Vision compares screenshot vs baseline (4 dimensions)
   d. Feedback: If score < threshold, refine code with previous scores as feedback
4. Loop exits when threshold met or max iterations reached
5. Workspace snapshot created after each iteration for historical replay

2. Technology Stack

Layer	Technology	Version	Purpose
Monorepo	pnpm workspaces	-	Package management
Backend	Fastify	4.x	HTTP API server
Backend	Node.js	18.18.1+	Runtime
Backend	TypeScript	5.x	Type safety
AI	@anthropic-ai/sdk	^0.39.0	Claude API client
AI Model	Claude Sonnet 4	claude-sonnet-4-20250514	Code generation & vision scoring
Screenshot	Playwright	^1.x	Headless Chromium captures
Frontend	Next.js	14.2.0	React framework
Frontend	React	18.3.0	UI library
Styling	Tailwind CSS	3.4.0	Utility-first CSS
UI Components	shadcn/ui	-	Radix UI primitives
Icons	lucide-react	^0.460.0	Icon library
Panels	react-resizable-panels	^2.1.0	Resizable pane layout
Validation	Zod	-	Schema validation
Template	Next.js + Tailwind + shadcn	-	Generated app template

3. Project Structure

```
AgentDOM/
├── apps/
│   ├── api/                        # Fastify backend (port 3001)
│   │   └── src/
│   │       ├── server.ts           # Entry point, route wiring
│   │       ├── lib/
│   │       │   ├── ids.ts          # ID generation (nanoid)
│   │       │   └── paths.ts        # Storage path helpers
│   │       ├── routes/
│   │       │   ├── projects.ts     # POST/GET /projects
│   │       │   ├── design-packs.ts # POST/GET /design-packs
│   │       │   ├── workspace.ts    # GET /files, /files/:path
│   │       │   ├── preview.ts      # Preview + historical endpoints
│   │       │   ├── runs.ts         # POST/GET /runs
│   │       │   ├── ws.ts           # WebSocket event stream
│   │       │   └── artifacts.ts    # Static file serving
│   │       └── services/
│   │           ├── project-service.ts
│   │           ├── design-pack-service.ts
│   │           ├── pack-validator.ts
│   │           ├── workspace-service.ts
│   │           ├── ai-executor.ts      # Core iteration loop
│   │           ├── prompt-builder.ts   # Claude prompt construction
│   │           ├── response-parser.ts  # XML response parsing
│   │           ├── screenshot-service.ts
│   │           ├── scoring-service.ts  # Claude Vision scoring
│   │           ├── run-service.ts      # Orchestration + trace tree
│   │           ├── snapshot-service.ts  # Workspace snapshots
│   │           ├── executor.ts         # Executor interface
│   │           └── stub-executor.ts    # Mock executor for testing
│   │
│   └── web/                        # Next.js frontend (port 3000)
│       └── src/
│           ├── app/
│           │   ├── layout.tsx      # Root layout (Inter font)
│           │   ├── page.tsx        # Redirect to /projects/demo
│           │   ├── globals.css     # Design tokens
│           │   └── projects/[projectId]/page.tsx
│           ├── components/
```

```
│           │   ├── layout/         # Main layout components
│           │   ├── filetree/       # File explorer
│           │   ├── trace/          # Agent trace tree
│           │   ├── common/         # Badge, Spinner, ErrorBanner
│           │   └── ui/             # Primitives (resizable, scroll, badge)
│           └── lib/
│               ├── api.ts          # REST client
│               ├── ws.ts           # WebSocket hook
│               ├── types.ts        # UI types + style maps
│               ├── utils.ts        # cn() utility
│               └── mock-data.ts    # Development mocks
│
├── packages/
│   ├── shared/                     # Shared types & schemas
│   │   └── src/
│   │       ├── index.ts            # Barrel exports
│   │       ├── schemas/
│   │       │   ├── manifest.zod.ts # Manifest validation
│   │       │   └── designIr.zod.ts # Design IR validation
│   │       └── contracts/
│   │           ├── api.ts          # REST API types
│   │           ├── events.ts       # Agent events & trace types
│   │           ├── ws.ts           # WebSocket message types
│   │           └── designPack.ts   # Re-exports
│   │
│   ├── sandbox-runner/             # Preview process manager
│   │   └── src/
│   │       ├── sandbox-manager.ts  # Process lifecycle + LRU
│   │       └── index.ts            # Barrel export
│   │
│   └── template-nextjs/            # Starter template
│       └── nextjs-tailwind-shadcn/ # Next.js + Tailwind + shadcn
│
├── storage/                        # Runtime data (gitignored)
│   └── projects/{projectId}/
│       ├── project.json
│       ├── workspace/
│       ├── snapshots/
│       ├── runtime/
│       └── artifacts/
```

```
│
└── templates/
    └── nextjs-tailwind-shadcn/     # Copied to new workspaces
```

4. Core Backend Logic

4.1 Server & Routing

File: apps/api/src/server.ts

The Fastify server initializes on port 3001 with CORS (localhost:3000), multipart upload support (50MB limit), and WebSocket. Two singletons are created at startup:

- SandboxManager — manages Next.js preview server processes (latest + historical)
- RunService — orchestrates AI-driven runs and maintains trace trees

Both are passed to route handlers via closure injection. Graceful shutdown handlers (SIGTERM/SIGINT) ensure all child processes are stopped before exit.

API Endpoints

Method	Path	Purpose
POST	/api/projects	Create a new project
GET	/api/projects/:id	Get project metadata
POST	/api/projects/:id/design-packs	Upload design pack (ZIP)
GET	/api/projects/:id/design-packs/:packId	Get design pack metadata
GET	/api/projects/:id/files	List workspace file tree
GET	/api/projects/:id/files/:path	Read file content
POST	/api/projects/:id/preview/start	Start latest preview server
POST	/api/projects/:id/preview/stop	Stop preview server
GET	/api/projects/:id/preview	Poll preview status
GET	/api/projects/:id/preview/latest	Get latest preview info
GET	/api/projects/:id/iterations	List iteration snapshots
POST	/api/projects/:id/iterations/:iter/preview	Start historical preview
GET	/api/projects/:id/iterations/:iter/preview	Poll historical preview status
POST	/api/projects/:id/runs	Start AI run
POST	/api/projects/:id/runs/stop	Stop active run
GET	/api/projects/:id/runs/report	Get run report with trace tree
GET	/api/projects/:id/ws	WebSocket event stream
GET	/api/projects/:id/artifacts/screenshots/:runId/:file	Serve screenshot PNG

GET	/api/projects/:id/design-packs/:packId/baselines/:path	Serve baseline PNG

4.2 Project & Design Pack Services

Project Service
File: apps/api/src/services/project-service.ts

Manages project lifecycle via filesystem-based metadata (project.json). On creation, it sets up the directory structure (workspace/, artifacts/design-packs/, artifacts/snapshots/, artifacts/reports/, artifacts/logs/) and copies the starter template into the workspace.

Design Pack Service
File: apps/api/src/services/design-pack-service.ts

Processes uploaded ZIP files through a pipeline:
1. Extract ZIP to design-packs/{packId}/
2. Remove macOS __MACOSX metadata folders
3. Unwrap single root folder (if ZIP contained one wrapper directory)
4. Validate manifest.json and design-ir.json via Zod schemas
5. Cross-validate: verify all baseline PNGs exist for every (target, breakpoint, state) combination
6. Collect baseline image paths recursively
7. Persist pack-meta.json with indexed paths

Pack Validator
File: apps/api/src/services/pack-validator.ts

Validates design pack structure using Zod schemas. Performs three levels of validation:
1. Schema validation: manifest.json against ManifestSchema, design-ir.json against DesignIrSchema
2. Referential integrity: manifest runDefaults.targetId must exist in targets array
3. Asset verification: baseline PNGs must exist for all target × breakpoint × state combinations
4. Cross-validation: IR targetIds must match manifest targetIds

4.3 AI Executor — The Iteration Loop

File: apps/api/src/services/ai-executor.ts

The AiExecutor is the core engine of the platform. It implements the iterative design-to-code refinement loop, coordinating code generation, screenshot capture, visual scoring, and feedback.

Execution Flow
1. Load Design Pack: Read manifest.json and design-ir.json, extract target config
2. Read Context: Load existing layout.tsx and globals.css from workspace

3. Iteration Loop (0 to maxIterations-1):

a. Code Generation:
- Build Claude prompt with Design IR, existing code, previous scores
- Call Claude API (claude-sonnet-4-20250514, 8192 max tokens)
- Parse XML response into file path/content pairs
- Write generated files to workspace

b. Screenshot Capture:
- Ensure preview server is running (start if needed)
- Warm up route: poll until non-404 response (handles file watcher lag)
- Launch Playwright Chromium at each breakpoint viewport
- Capture viewport screenshots as PNG files

c. Visual Scoring:
- For each breakpoint, compare screenshot vs baseline using Claude Vision
- Score 4 dimensions: layout, style, a11y, perceptual
- Aggregate scores across breakpoints (average each dimension)
- Calculate delta from previous iteration

d. Snapshot & Threshold Check:
- Create tar.gz workspace snapshot for historical preview
- If score >= threshold: exit loop (success)
- If max iterations reached: exit loop
- Otherwise: pass scores as feedback for next iteration

Event Emission

Every step emits AgentEvent objects with unique IDs, timestamps, and payloads. Event types: nodeCreated, nodeStarted, nodeProgress, nodeFinished, nodeFailed, artifactAdded. These events build a hierarchical trace tree for real-time UI visualization.

Abort Support

The executor uses AbortController for cancellation. The stop() method sets a stopped flag and aborts the controller. Each major step checks the flag before proceeding, enabling responsive cancellation mid-iteration.

4.4 Prompt Builder & Response Parser

Prompt Builder

File: apps/api/src/services/prompt-builder.ts

Constructs Claude prompts for code generation. The system prompt defines:
- Tech stack: Next.js 14 App Router, React 18, Tailwind CSS 3, shadcn/ui, lucide-react

• Output format: XML with <files><file path="...">content</file></files>
• Key rules: "use client" only for interactive components, bbox → flexbox/grid (not absolute), use shadcn color variables, valid TypeScript/TSX only

The user prompt includes:
• Route and primary breakpoint info
• Formatted Design IR nodes (name, importance, layout targets, style targets, a11y targets)
• Existing layout.tsx and globals.css content
• On iteration > 0: feedback section with previous scores, weak areas, previous code

Response Parser
File: apps/api/src/services/response-parser.ts

Parses Claude's XML-structured code response into file path/content pairs.
• Strips markdown code fences
• Extracts <files>...</files> block
• Extracts each <file path="...">...</file> entry via regex
• Security: rejects paths with ".." or leading "/"
• Normalizes: prepends "src/" if not already present
• Trims blank lines but preserves indentation

4.5 Screenshot Service (Playwright)

File: apps/api/src/services/screenshot-service.ts

Captures screenshots of rendered pages at multiple breakpoints using Playwright.

1. Launch Chromium browser (headless, --no-sandbox, --disable-gpu)
2. For each breakpoint:
   • Create browser context with viewport size and device scale factor
   • Navigate to target URL with "networkidle" wait strategy
   • Wait 500ms for layout stability (post-hydration)
   • Capture viewport as PNG (fullPage: false)
   • Store file size metadata
3. Close browser
4. Return array of {breakpointId, filePath, sizeBytes}

4.6 Visual Scoring Service (Claude Vision)

File: apps/api/src/services/scoring-service.ts

Compares generated screenshots against design baselines using Claude Vision, scoring across 4 independent dimensions:

Dimension	Weight	What It Measures
Layout	0.30	Element positioning, sizing,

		spacing, alignment, responsive behavior
Style	0.30	Colors, typography, borders, shadows, visual styling accuracy
A11y	0.20	Semantic elements, ARIA attributes, interactive element visibility
Perceptual	0.20	Overall visual similarity (human at-a-glance impression)

The scoring prompt sends both baseline and screenshot as base64-encoded images to Claude Vision, along with Design IR node context (nodeId, name, importance, layout/style/a11y targets). Claude returns a JSON object with scores from 0.0 to 1.0 for each dimension. The overall score is computed as a weighted average: 0.3×layout + 0.3×style + 0.2×a11y + 0.2×perceptual.

Scores are aggregated across breakpoints by averaging each dimension. The aggregate includes per-breakpoint breakdown for detailed analysis.

4.7 Run Service & Event System

File: apps/api/src/services/run-service.ts

Orchestrates runs and maintains trace trees. Extends EventEmitter to broadcast events to WebSocket subscribers.

Run State

```
interface RunState {
  runId: string;
  projectId: string;
  packId: string;
  status: TraceStatus;    // queued | running | success | error
  tree: IterationNode | null;  // hierarchical trace tree
  startedAt: string;
}
```

Trace Tree Building (applyEvent)

The applyEvent function builds a hierarchical tree incrementally from AgentEvent objects:

- Creates root node from the first event
- Subsequent events: find node by nodeId, update in place
- Parent-child relationships inferred from nodeId hierarchy
  (e.g., "root-iter0-codegen" has parent "root-iter0")

• Status transitions: nodeStarted → running, nodeFinished → success/error
• Artifacts accumulated in node.artifacts array
• Deep clone on update to maintain React state immutability

Node ID Scheme

```
root                        # Run root
root-iter0                  # Iteration 0
root-iter0-codegen          # Code generation step
root-iter0-screenshot       # Screenshot step
root-iter0-screenshot-desktop  # Desktop breakpoint
root-iter0-scoring          # Scoring step
root-iter1                  # Iteration 1
...
```

Event Buffering

RunService buffers all events for each project. When a new WebSocket client connects, all buffered events are replayed to bring the client up to speed with the current run state. This supports late-joining observers and page refreshes during active runs.

4.8 Sandbox Manager (Preview Servers)

File: packages/sandbox-runner/src/sandbox-manager.ts

Manages lifecycle of Next.js dev server processes. Maintains two separate Maps:
• previews: Map<projectId, PreviewProcess> — one latest preview per project
• historicalPreviews: Map<key, PreviewProcess> — keyed by "{projectId}:iter:{N}"

Process Lifecycle

1. Find free port via OS-level socket binding
2. Self-heal: if workspace lacks package.json, copy from template
3. Install: run "npm install --include=dev" if node_modules missing
4. Spawn: run "next dev --port {port}" with detached process group
5. Detect ready: listen for "Ready in" or "Local:" in stdout/stderr
6. Timeout: 120 seconds max for startup
7. Kill: SIGTERM to process group, SIGKILL after 5 seconds if still alive

Resource Management

Policy	Value	Description
Latest TTL	30 minutes	Idle timeout before automatic reaping
Historical TTL	10 minutes	Shorter TTL for historical previews
Max Historical	2	LRU eviction when at capacity
Reaper Interval	60 seconds	Background cleanup check

		frequency
Startup Timeout	120 seconds	Max wait for dev server ready

Environment Isolation

Child processes run with a cleaned environment:

- NODE_OPTIONS stripped (prevents tsx preload conflicts)
- NODE_ENV set to "development"
- PATH prepended with /usr/local/bin, pnpm path
- Process group detached for clean SIGTERM handling

4.9 Snapshot Service (Historical Previews)

File: apps/api/src/services/snapshot-service.ts

Creates and manages workspace snapshots for historical preview access.

createSnapshot(): Archives workspace as tar.gz, excluding node_modules, .next, .git. Writes sidecar JSON metadata (iterationIndex, createdAt, archivePath).

extractSnapshot(): Extracts tar.gz to runtime directory. Idempotent — returns immediately if already extracted. Used when starting a historical preview server.

Storage layout:

```
storage/projects/{id}/snapshots/iter-{N}.tar.gz  (archive)
storage/projects/{id}/snapshots/iter-{N}.json    (metadata)
storage/projects/{id}/runtime/iter-{N}/workspace/ (extracted for preview)
```

4.10 WebSocket Streaming

File: apps/api/src/routes/ws.ts

Provides real-time event streaming via WebSocket at GET /api/projects/:projectId/ws.

On connection:

1. Accept WebSocket upgrade
2. Replay buffered events for late-joining clients
3. Forward agentEvent, runStarted, runFinished events as ServerWsMessage objects
4. Handle ping messages
5. Clean up listeners on close

Message types sent to client:

- { type: "agentEvent", event: AgentEvent } — step progress
- { type: "runStarted", runId, projectId } — run lifecycle

• { type: "runFinished", runId, projectId, status } — run completion
• { type: "error", message } — server error

4.11 Shared Types & Contracts

Package: packages/shared/

Manifest Schema (manifest.zod.ts)

Defines design pack structure:
• Breakpoints: viewport width/height with device scale factor
• States: UI states (e.g., "default", "hover")
• Targets: route targets with file entry hints
• RunDefaults: threshold (0-1), max iterations, target to run
• Referential integrity: runDefaults.targetId must exist in targets array

Design IR Schema (designIr.zod.ts)

Visual specification that Claude uses to generate code:
• IRNode: Visual element with bounding box, tolerance, layout targets, style targets, a11y targets
• LayoutTargets: Bbox with position/size tolerances
• StyleTargets: Font, color (RGB strings), spacing, shadows
• A11yTargets: ARIA roles, labels, landmark info
• ComponentMapping: Design intent hints (component name + props)
• matchImportance: "critical" | "normal" | "low" — used by scoring

Event Types (events.ts)

• IterationNode: Hierarchical trace tree node with:
  - nodeId, stepKey, title, status, message
  - startedAt/finishedAt timestamps
  - iterationIndex for iteration-level nodes
  - score: { overall, layout, style, a11y, perceptual, deltaFromPrev }
  - artifacts: ArtifactLink[] (downloadable files)
  - children: IterationNode[] (sub-steps)

• AgentEvent: Event emitted during execution
  - eventId, type, projectId, packId, nodeId, ts, payload
  - Types: nodeCreated, nodeStarted, nodeProgress, nodeFinished, nodeFailed, artifactAdded

• ArtifactLink: { id, kind, label, href, mime?, sizeBytes? }
• ArtifactKind: designPack | baselineImage | designIr | workspaceFile | snapshotJson | snapshotImage | scoreReportJson | scoreReportHtml | diffPatch | log | previewUrl

5. Core Frontend Logic

5.1 App Layout & Routing

The Next.js app uses the App Router with a minimal route structure:

• / — Redirects to /projects/demo
• /projects/[projectId] — Dynamic project workspace page

Root layout sets the Inter font, "Vibe Studio" metadata, and imports globals.css which defines the CSS variable design token system. The entire app is a fullscreen layout with no scrolling on the body (overflow: hidden).

5.2 ProjectWorkspace (State Root)

File: apps/web/src/components/layout/ProjectWorkspace.tsx

The central state management component. All application state lives here and flows down via props. Key state categories:

Project State

• projectId: Auto-created for placeholder slugs ("demo", "new") via API
• projectName, activePackId: Set after design pack upload
• targetRoute: Default route from manifest (e.g., "/")
• fileTreeRefreshKey: Incremented to trigger file tree reload

Run State

• runActive: Boolean, true while AI run is in progress
• previewAutoStart, previewRefreshKey: Control preview lifecycle during runs

Preview State

• previewMode: "latest" | "iteration" — which preview is showing
• pinnedIterationId: Which iteration is pinned (or null)
• overridePreviewUrl: Direct URL for historical preview iframe
• isFullscreen, fullscreenUrl: Fullscreen overlay state

Key Handlers

• handleDesignPackUpload(): Upload ZIP → validate → set active pack → refresh file tree
• handleRun(): Start AI run → reset preview to latest mode
• handleRunComplete(): Run finished → refresh file tree → auto-start preview
• handleIterationClick(): POST to start historical preview → poll until ready → set override URL
• handleRefreshLatest(): Clear override URL → reset to latest mode → refresh preview
• handleFullscreen(): Get current preview URL → set fullscreen overlay
• handleArtifactClick()/handleCloseArtifact(): Manage artifact viewer tab

5.3 Three-Pane Layout

File: apps/web/src/components/layout/ThreePaneLayout.tsx

Uses react-resizable-panels for a horizontal three-pane layout:

• Left Panel (25%, 15-40% range): FileTreePane — file explorer + code viewer
• Center Panel (50%, 30%+ min): CenterPane — preview + baseline + artifact tabs
• Right Panel (25%, 15-40% range): TracePane — real-time agent trace tree

Panels are separated by draggable ResizableHandle components with visible grip icons. All panels within a group maintain their proportions as the window resizes.

5.4 Preview Pane & Dev Server Lifecycle

File: apps/web/src/components/layout/PreviewPane.tsx

Manages the live preview iframe and dev server lifecycle. Operates in two modes:

Normal Mode

• Sends POST to start preview server
• Polls GET /preview every 2 seconds until ready
• Renders iframe with sandbox="allow-scripts allow-same-origin"
• Auto-start support: triggers on first mount if autoStart=true
• Auto-refresh: reloads iframe when refreshKey changes (2s delay for HMR)

Override Mode (Historical Preview)

• When overridePreviewUrl is set, bypasses polling entirely
• Renders amber "Historical" badge with the override URL
• Direct iframe rendering without lifecycle management

UI States

Status	Display	Icon
stopped	Start button centered	Monitor
installing	Loading indicator	Package (pulsing)
starting	Loading indicator	Zap (pulsing)
ready	Iframe with URL bar	Green dot
error	Error message + Retry	Monitor (red)

5.5 Agent Trace Visualization

Files: TracePane.tsx, TraceTree.tsx, TraceNode.tsx

Renders a recursive tree of trace nodes representing the run execution. Each node shows status, duration, score, and artifacts.

TraceNode Features

• Expand/Collapse: Chevron toggle for child visibility
• Status Indicator: Spinner (running) or colored dot (queued/success/error/skipped)

• Title + Status Badge: Node name with colored status chip
• Score Display: Overall score as percentage, with per-dimension breakdown (Layout, Style, A11y) and delta from previous iteration (color-coded +/-)
• Artifact Buttons: Clickable badges with download icon for each artifact
• Duration: Formatted as "XXms" or "X.Xs"
• Iteration Click: Completed iteration nodes are clickable (Eye icon hint) to trigger historical preview via onIterationClick callback
• Recursive: Children rendered with depth+1 for indentation (8 + depth×16 px)

5.6 File Tree Explorer

Files: FileTreePane.tsx, FileTree.tsx, FileViewer.tsx

Left panel with vertical split: file tree (top 40%) and code viewer (bottom 60%).

FileTree: Recursive tree with expand/collapse, auto-expands directories up to depth 2. Files show status indicators (A=added green, M=modified yellow, U=untracked gray). Clicking a file fetches its content via API.

FileViewer: Displays selected file content in monospace <pre><code> block with file path header and language badge. No syntax highlighting (plain text rendering).

5.7 Baseline Image Viewer

File: apps/web/src/components/layout/BaselinePane.tsx

Displays baseline reference images from the uploaded design pack.
• Loads baselines when packId changes via API
• Parses path format: baselines/{targetId}/{breakpointId}/{stateId}.png
• Shows breakpoint selector dropdown if multiple baselines exist
• Images displayed centered with object-contain, light background, subtle shadow

5.8 Historical Preview & Click-to-Focus

When a user clicks a completed iteration in the Agent Trace tree:

1. handleIterationClick fires in ProjectWorkspace
2. Sets previewMode to "iteration" and pinnedIterationId
3. Sends POST /iterations/:id/preview to start historical preview server
4. If not immediately ready, polls GET /iterations/:id/preview every 2 seconds
5. CenterPane shows loading overlay: "Starting historical preview..."
6. When ready, sets overridePreviewUrl → PreviewPane renders historical iframe
7. CenterPane header shows amber "Pinned: Iteration N" chip
8. Clicking "Refresh (Latest)" button resets to latest preview mode

The polling has a 3-minute timeout (90 polls × 2 seconds). The backend handles npm install + next dev startup for the extracted snapshot workspace.

5.9 Fullscreen Preview

File: apps/web/src/components/layout/FullscreenPreview.tsx

Fixed overlay (z-50) covering the entire viewport with a black background.
• Full-size iframe with the preview URL
• Exit button: top-right with Minimize2 icon and backdrop blur
• ESC key handler via useEffect + keydown listener
• Works in both latest and iteration preview modes

5.10 WebSocket Hook (useTraceStream)

File: apps/web/src/lib/ws.ts

Custom React hook that manages WebSocket connection and trace tree state.

Returns: { tree: IterationNode | null, runStatus: "idle" | "running" | "success" | "error" }

• Connects to ws://localhost:3001/api/projects/{projectId}/ws
• Resets tree when runActive transitions false → true (new run)
• Applies incoming agentEvent messages to build trace tree incrementally
• Uses deep clone + mutation pattern for React state immutability
• findNode(): Recursive tree search by nodeId
• applyEvent(): Maps event types to tree mutations (create, update status, add artifacts)
• Parent-child inference: strips last "-segment" from nodeId to find parent
• Cleans up WebSocket on unmount or when runActive becomes false

5.11 API Client

File: apps/web/src/lib/api.ts

REST API client using native fetch(). All functions are async and throw on HTTP errors.

Functions:
• createProject(name) — POST /projects
• uploadDesignPack(projectId, file) — POST /design-packs (FormData)
• getDesignPack(projectId, packId) — GET /design-packs/:id
• getBaselineUrl(projectId, packId, path) — URL builder
• getFileTree(projectId) — GET /files
• getFileContent(projectId, path) — GET /files/:path
• startPreview / stopPreview / getPreviewStatus — preview lifecycle
• startRun / stopRun / getRunReport — run lifecycle
• getIterations — GET /iterations (list snapshots)

- startHistoricalPreview — POST /iterations/:id/preview
- getHistoricalPreviewStatus — GET /iterations/:id/preview (poll)
- getLatestPreview — GET /preview/latest

6. Design Elements

6.1 Design Token System

All colors are defined as CSS custom properties in HSL format, enabling light/dark mode switching via class-based toggling. Colors are consumed through Tailwind utilities like bg-primary, text-muted-foreground, etc.

Light Mode Colors

Token	HSL Value	Usage
--background	0 0% 100%	Page background (white)
--foreground	222.2 84% 4.9%	Primary text (near-black)
--primary	222.2 47.4% 11.2%	Primary actions, buttons
--primary-foreground	210 40% 98%	Text on primary backgrounds
--secondary	210 40% 96.1%	Secondary backgrounds
--muted	210 40% 96.1%	Muted backgrounds, cards
--muted-foreground	215.4 16.3% 46.9%	Secondary text, labels
--accent	210 40% 96.1%	Hover states, highlights
--destructive	0 84.2% 60.2%	Error states, delete actions
--border	214.3 31.8% 91.4%	Default borders (light gray)
--ring	222.2 84% 4.9%	Focus ring color
--radius	0.5rem	Base border radius (8px)

Dark Mode Colors

Dark mode inverts the color scheme: primary becomes light (210 40% 98%), background becomes dark (222.2 84% 4.9%), accent becomes slightly darker (217.2 32.6% 17.5%). Toggled via "dark" class on html element.

6.2 Component Library

UI Primitives (shadcn/ui)

- ResizablePanelGroup / ResizablePanel / ResizableHandle: Wrapper for react-resizable-panels with direction-aware styling and optional grip icon handles
- ScrollArea / ScrollBar: Radix UI scroll area with styled thumb and touch support
- Badge: CVA-based badge with variants (default, secondary, destructive, outline)

Common Components

- StatusBadge: Maps TraceStatus to colored badge (inline-flex, rounded-full)
  - queued: gray, running: blue, success: green, error: red, skipped: gray
- Spinner: Lucide Loader2 with animate-spin class, configurable size
- ErrorBanner: Red alert with AlertTriangle icon and dismissal button

6.3 Layout Patterns

Full-screen application layout (h-screen w-screen, overflow-hidden):

• ProjectHeader: Fixed height top bar with border-bottom
• ThreePaneLayout: flex-1 horizontal split with resizable panels
  - Left (25%): Vertical split — FileTree (40%) | FileViewer (60%)
  - Center (50%): Tabs — Preview | Baseline | Artifact
  - Right (25%): Single pane — Agent Trace
• FullscreenPreview: Fixed overlay (z-50, inset-0) when active

Pane Headers: Consistent pattern of flex row with icon + label (text-xs uppercase tracking-wider), border-bottom, muted foreground color.

Panel Preservation Strategy

Preview and Baseline panes stay mounted but invisible (using CSS "invisible" class) when not the active tab. This preserves iframe state during tab switches — the dev server connection is not interrupted when switching to Baseline and back.

6.4 Typography & Iconography

Typography

• Font: Inter (Google Fonts)
• text-xs (12px): Labels, badges, timestamps, secondary info
• text-sm (14px): Body text, file content, trace messages
• text-base (16px): Section headers, empty state messages
• Font weights: normal (400), medium (500), semibold (600), bold (700)
• Uppercase tracking: "text-xs font-semibold uppercase tracking-wider" for section labels

Iconography (Lucide)

All icons from lucide-react, used at consistent sizes:
• 10px: Close (X) on tabs, FileDown on artifacts
• 12px: Inline controls (refresh, stop, Eye hint)
• 14px: Tab icons, status dots, trace chevrons, header controls
• 16px: Action buttons (upload, minimize)
• 32px: Loading states (Package, Zap), empty states (Activity)
• 48px: Large empty states (Monitor)

Key icons:
• Upload, Package, Play, Square — Header controls
• Monitor, Image, Camera — Tab labels
• RefreshCw, Maximize2, Minimize2 — Preview controls
• Activity, ChevronRight/Down — Trace tree
• Folder/FolderOpen, FileText — File tree

• Loader2 — Spinner (with animate-spin)
• Eye — Clickable iteration hint

6.5 Status Indicators

Trace Status Colors

Status	Badge	Dot	Usage
queued	bg-gray-200 text-gray-700	bg-gray-400	Waiting to start
running	bg-blue-100 text-blue-700	bg-blue-500	Currently executing
success	bg-green-100 text-green-700	bg-green-500	Completed successfully
error	bg-red-100 text-red-700	bg-red-500	Failed with error
skipped	bg-gray-100 text-gray-500	bg-gray-300	Skipped step

Preview Status Indicators

• Green dot + "Live": Preview server ready and serving
• Amber dot + "Historical": Showing historical iteration preview
• Amber badge "Pinned: Iteration N": When viewing pinned iteration
• Blue pulsing dot + "Live": WebSocket actively receiving trace events
• Animated icons: Package (pulsing) during install, Zap (pulsing) during startup, Loader2 (spinning) for general loading

File Status Indicators

• "A" (green-600): File added to workspace
• "M" (yellow-600): File modified
• "U" (gray-500): Untracked file

Score Display

• Overall score shown as percentage (e.g., "85%")
• Per-dimension breakdown: "Layout: 90% Style: 78% A11y: 92%"
• Delta from previous iteration: "+5%" (green-600) or "-3%" (red-600)
• Scores displayed in trace nodes at right edge

7. Data Flow & Integration

7.1 Design Pack Upload Flow

```
User drags ZIP onto ProjectHeader
  → handleDesignPackUpload(file)
    → POST /api/projects/{id}/design-packs (FormData)
      → Extract ZIP → Remove __MACOSX → Unwrap root folder
      → Validate manifest.json (Zod + referential integrity)
      → Validate design-ir.json (Zod + cross-validation)
      → Verify baseline PNGs exist for all target × breakpoint × state
      → Collect baseline image paths → Write pack-meta.json
    ← { packId, projectId, defaultRoute, validationErrors? }
  → Set activePackId, targetRoute
  → Refresh file tree (fileTreeRefreshKey++)
  → Show success/error banner
```

7.2 Run Execution Flow

```
User clicks "Run" button
  → handleRun() → POST /api/projects/{id}/runs { packId }
    → RunService.startRun(projectId, packId, workspacePath)
      → Create AiExecutor instance
      → Wire executor events:
          "agentEvent" → applyEvent() (build tree) → buffer → emit to WS
          "done" → emit "runFinished"
      → executor.start(options)
        → AiExecutor.run() [async fire-and-forget]
          → Load manifest + Design IR
          → Read existing layout.tsx + globals.css
          → Loop iterations:
            1. CodeGen: buildPrompt → Claude API → parseXML → write files
            2. Screenshot: waitForPreview → warmUp → Playwright capture
            3. Score: Claude Vision × N breakpoints → aggregate
            4. Snapshot: tar.gz workspace
            5. Check threshold → continue or break
          → emit "done"
      → Emit "runStarted"
    ← { runId, projectId, status: "running" }
  → setRunActive(true)
  → TracePane: useTraceStream connects WebSocket
    → Receives agentEvent messages → builds trace tree → re-renders
```

7.3 Real-time Event Streaming

```
Backend (RunService)                    Frontend (useTraceStream)
       │                                         │
       │  ← WebSocket connect ←                  │
       │                                         │
       │  → Replay buffered events →              │ Apply each event to tree
       │                                         │
       │  → agentEvent (nodeCreated) →            │ Create node in tree
       │  → agentEvent (nodeStarted) →            │ Set status: "running"
       │  → agentEvent (nodeProgress) →           │ Update message
       │  → agentEvent (artifactAdded) →          │ Add artifact to node
       │  → agentEvent (nodeFinished) →           │ Set status + score
       │  → runFinished →                         │ Set runStatus: "success"
       │                                         │
       │  ← Close ←                               │ Cleanup
```

7.4 Historical Preview Flow

```
User clicks completed iteration in TraceNode
  → onIterationClick(iterationIndex)
    → handleIterationClick(iterationIndex)
      → setPreviewMode("iteration"), setPinnedIterationId(N)
      → setOverridePreviewUrl(null) [show loading overlay]
      → POST /api/projects/{id}/iterations/{N}/preview
        → extractSnapshot(projectId, N) [tar xzf → runtime dir]
        → sandboxManager.startHistoricalPreview(projectId, N, path)
          → Evict LRU if > 2 historical previews
          → npm install + next dev on extracted workspace
        ← { previewUrl, status: "installing", iterationId }
      → Poll GET /iterations/{N}/preview every 2s
        → Status: installing → starting → ready
      → setOverridePreviewUrl(previewUrl) [clear loading overlay]
    → PreviewPane renders override iframe with "Historical" badge
    → CenterPane shows "Pinned: Iteration N+1" chip

User clicks "Refresh (Latest)" button
  → handleRefreshLatest()
    → setPreviewMode("latest"), setPinnedIterationId(null)
    → setOverridePreviewUrl(null) [back to normal preview]
    → Increment previewRefreshKey → iframe reloads
```

8. Key Configuration Values

Config	Value	Purpose
API Port	3001	Backend server port
Frontend Port	3000	Next.js dev server port
CORS Origin	http://localhost:3000	Allowed frontend origin
Multipart Limit	50 MB	Design pack upload max size
Claude Model	claude-sonnet-4-20250514	Used for codegen + scoring
Codegen Max Tokens	8192	Max output for code generation
Scoring Max Tokens	1024	Max output for score JSON
Score Weights	L:0.3 S:0.3 A:0.2 P:0.2	Weighted average formula
Default Threshold	0.92	Score to stop iterating
Default Max Iterations	10	Max refinement loops
Latest Preview TTL	30 minutes	Idle timeout before reap
Historical Preview TTL	10 minutes	Shorter TTL for snapshots
Max Historical Previews	2	LRU eviction limit
Reaper Interval	60 seconds	Cleanup check frequency
Dev Server Timeout	120 seconds	Max wait for "Ready in"
Route Warmup Timeout	30 seconds	Max wait for non-404
Historical Poll Max	90 × 2s = 3 min	Frontend polling timeout
Screenshot Settle	500ms	Wait after networkidle
HMR Buffer	2 seconds	Wait before iframe refresh

9. File Inventory

Backend (apps/api/src/)

File	Lines	Purpose
server.ts	~80	Entry point, route wiring, graceful shutdown
lib/ids.ts	~10	ID generation (nanoid)
lib/paths.ts	~50	Storage path helpers
routes/projects.ts	~50	Project CRUD endpoints
routes/design-packs.ts	~80	Design pack upload + metadata
routes/workspace.ts	~40	File tree + content endpoints
routes/preview.ts	~160	Preview + iterations + historical endpoints
routes/runs.ts	~60	Run start/stop/report endpoints
routes/ws.ts	~60	WebSocket event streaming
routes/artifacts.ts	~70	Screenshot + baseline static file serving
services/project-service.ts	~70	Project lifecycle (create, read, update)
services/design-pack-service.ts	~120	ZIP extraction, validation, indexing
services/pack-validator.ts	~100	Zod validation + cross-checks
services/workspace-service.ts	~80	File browsing + language detection
services/ai-executor.ts	~440	Core iteration loop (codegen → screenshot → score)
services/prompt-builder.ts	~180	Claude prompt construction
services/response-parser.ts	~70	XML response parsing
services/screenshot-service.ts	~80	Playwright screenshot capture
services/scoring-service.ts	~180	Claude Vision scoring (4 dimensions)
services/run-service.ts	~160	Run orchestration + trace tree
services/snapshot-service.ts	~130	Workspace snapshot create/extract
services/executor.ts	~30	Executor interface definition
services/stub-executor.ts	~80	Mock executor for testing

Frontend (apps/web/src/)

File	Lines	Purpose
app/layout.tsx	~25	Root layout with Inter font

app/page.tsx	~5	Redirect to /projects/demo
app/globals.css	~80	Design tokens (CSS variables)
app/projects/[projectId]/page.tsx	~15	Dynamic project page
lib/api.ts	~205	REST API client
lib/ws.ts	~120	WebSocket hook (useTraceStream)
lib/types.ts	~30	UI types + style mappings
lib/utils.ts	~6	cn() utility
components/layout/ProjectWorkspace.tsx	~265	State root, all handlers
components/layout/ThreePaneLayout.tsx	~95	Three-pane resizable layout
components/layout/ProjectHeader.tsx	~110	Top toolbar with upload + run
components/layout/FileTreePane.tsx	~60	File tree + viewer split
components/layout/CenterPane.tsx	~170	Preview/baseline/artifact tabs
components/layout/PreviewPane.tsx	~275	Live preview iframe + lifecycle
components/layout/BaselinePane.tsx	~100	Baseline image viewer
components/layout/TracePane.tsx	~60	Agent trace wrapper
components/layout/FullscreenPreview.tsx	~47	Fullscreen overlay
components/filetree/FileTree.tsx	~130	Recursive file tree
components/filetree/FileViewer.tsx	~50	Code display
components/trace/TraceTree.tsx	~43	Trace tree root
components/trace/TraceNode.tsx	~170	Recursive trace node
components/common/Badge.tsx	~25	Status badge
components/common/Spinner.tsx	~15	Loading spinner
components/common/ErrorBanner.tsx	~30	Error message banner
components/ui/resizable.tsx	~60	Resizable panel wrappers
components/ui/scroll-area.tsx	~50	Radix scroll area
components/ui/badge.tsx	~40	CVA badge variants

Shared Packages

File	Purpose
packages/shared/src/index.ts	Barrel exports for all shared types
packages/shared/src/schemas/manifest.zod.ts	Manifest validation schema
packages/shared/src/schemas/designIr.zod.ts	Design IR validation schema
packages/shared/src/contracts/api.ts	REST API response types
packages/shared/src/contracts/events.ts	Agent events + trace tree types
packages/shared/src/contracts/ws.ts	WebSocket message types
packages/shared/src/contracts/designPack.ts	Design pack type re-exports
packages/sandbox-runner/src/sandbox-manager.ts	Preview process lifecycle
packages/sandbox-runner/src/index.ts	Barrel export